1. ECDLP dataset models: development pipeline and pilot (2026-11-02)

Status: development-only. Under `docs/ml-dataset/training-protocol-20261101.md`, development data may tune parsers and debug a model. It may not select a winner, estimate generalisation, or provide a final test set. Nothing below does any of those.

Study: `research/ecdlp_dataset_models_20261102` · **protocol frozen before data:** `655f52ecf` (`dev-pilot-protocol.json`, canonical sha256 `5740dd6d…`) · **plan:** `dev-pilot-plan.json` (960 cells) · **generator:** `crypto dataset generate` built from `891de41ad`, clean provenance · **host:** session container, 4 vCPU, $0 · **evidence:** `results/`.

1.1. What was built

`training-protocol-20261101` preregisters how models on the native ECDLP/PDP dataset must be trained and evaluated, but no training or evaluation code existed. This study adds a native-Rust evaluator and trainer (`ecdlp-dataset-models`) for the protocol's **resource/success model**: the probability that an attempted arm reaches its event within the fixed work cap, given public instance features.

- **`audit`** scans every public export file. It flags a forbidden key inside `inputs`, and any restricted known-answer or recovered scalar written as a literal anywhere in the public bytes. It also writes the allowlist of input paths present.
- **`join`** is the evaluator side. It joins restricted labels only after featurising public inputs (21 engineered descriptors). The attempted arm enters as a one-hot **query** input, chosen before any outcome exists. It has two endpoints:
  - **completion** (primary): the arm completed with a verified result within the cap. `budget_exhausted`, `timeout` and `partial` are right-censored at the cap, so they count as 0.
  - **ecdlp**: the scalar was recovered without the witness and independently replayed.

  `unsupported` and `error` are excluded and counted.
- **`train`** evaluates leave-one-component-out over the exporter's `split_component`. It fits a stratum-majority baseline (arm × family × m), logistic regression and a 32-unit MLP, and reports Brier score, log loss, accuracy and calibration, pooled and per arm. A within-stratum label permutation serves as the negative control.
- **Positive controls:** seven unit tests, plus a leak-canary column equal to the label on the real data.

1.2. Pilot data (measured)

960 cells: n ∈ {23, 25, 27, 29, 31}, m ∈ {2, 3}, Koblitz, binary and prime families, both twists, two curve seeds, two keys, and four arms (`enumerate`, `bsgs`, `root`, `double-large-prime`), at work limit 100 000. All 960 receipts were written and their artifact hashes verify.

Outcome counts: 665 `verified`, 96 `partial`, 51 `budget_exhausted`, 8 `timeout`, 120 `unsupported`, 20 `error`. The generator's status reports 425 cells as independently solved ECDLP instances.

- **The 20 errors** are all `root` runs on prime-field curves with m = 2, at n = 25 (1), 27 (6), 29 (5) and 31 (8). In each, the solver trace exceeded the protocol's 64 MiB capture budget (`StreamByteLimit`).
- **One extra exclusion:** a `double-large-prime` timeout was exported without a feature row (`missing_verified_large_prime_domain_or_transcript`).
- **Training rows:** 819 (665 positive under completion, 425 under ecdlp).

Leakage audit: 479 public feature rows, 94 restricted scalars checked, 0 forbidden key paths, 0 scalars found: pass. The packed generation log was also checked separately. No restricted scalar of four or more digits appears in it as a quoted value; 25 scalars are under four digits and too short for a literal search to mean anything.

1.3. Results (development metrics, leave-one-field-size-out, 5 components)

table	majority	logistic regression	MLP	perm. majority	perm. logreg	perm. MLP
completion	0.0603	**0.0205**	0.0352	0.0527	0.0528	0.0565
ecdlp	0.0603	**0.0203**	0.0276	0.0527	0.0528	0.0639
completion + leak canary	0.0603	0.0001	0.0000	0.0527	0.0538	0.0576

Pooled Brier score (lower is better), n = 819. Accuracy under completion: majority 0.934, logistic regression 0.972, MLP 0.950. The "perm." columns are the same models trained and tested on labels permuted within each stratum.

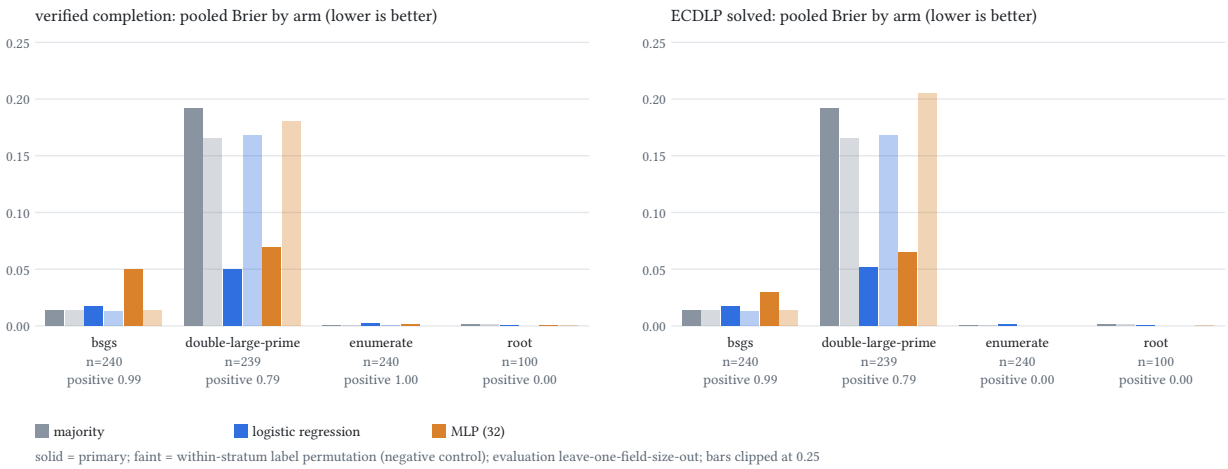


Figure 1: Per-arm pooled Brier score: three models, primary (solid) against the within-stratum permutation control (faint). Values from `results/summary-{completion,ecdlp}.json`.

1.4. Reading the results

- **The arm nearly fixes the label.**
  - `root` is censored on all 100 rows: every run is `partial`, and higher-m elimination is unsupported.
  - `enumerate` completes on all 240 rows and recovers the scalar on none. It is a PDP enumeration whose verified result is often "no decomposition exists".
  - `bsgs` solves 237 of 240; its 3 failures are n = 29 timeouts.

  This is also why the stratum majority scores exactly the same under both endpoints. Switching endpoints flips the whole `enumerate` stratum from 1 to 0, and a per-stratum rate is equally right either way. The pooled Brier score is mostly arm bookkeeping.
- **The only arm with something to learn is `double-large-prime`** (188 of 239 completed). There, logistic regression scores 0.050 against 0.192 for the majority baseline. The permutation control removes that gain (0.168), so public instance features carry signal within this arm in the pilot.
  - Failures do not rise steadily with n:
    - binary: fails at n = 23, is clean at n = 25–27, and fails again at n = 29–31;
    - Koblitz: fails only at n = 23;
    - prime: fails at n = 27–31.
  - With two curve seeds per cell, failures come in groups of two or four, consistent with individual curves rather than field size. So the signal is plausibly curve structure (subgroup order, cofactor, factor-base shape).
  - That is a hypothesis for the decisive run, not a finding: here, 239 rows from 60 curves.
- **The controls behave.**
  - The leak canary is learned almost perfectly (Brier ≤ 0.0001), so the evaluator would show a leak.
  - Under permutation the canary no longer matches the label, and every model falls back to the stratum rate.
  - The audit's planted-scalar and forbidden-key tests fail as designed (unit tests).

1.5. Bugs found by the controls

- **Trainer.** A feature that was constant inside a training fold was standardised with a floored deviation, so held-out values exploded, and logistic regression predicted 0 everywhere even with the canary column. Such features now contribute zero. This was caught by the canary before any pilot data existed.
- **Label definition.** The first 118 pilot cells showed 83 `verified` but only 53 solved, because of the `enumerate` behaviour above. The **ecdlp** endpoint was added then, before training. Completion remains the primary label. **This endpoint was not in the frozen protocol.**

1.6. Execution record and deviations

- **Where it ran.** The EKS route was unavailable because this session has no AWS identity. Generation ran directly in the session container, not through an experiment-director task card. This is a development pilot for debugging the pipeline, so no measurement claim depends on how it was executed.
- **Disk incident.** While generation was running, an unrelated full git checkout filled the disk for under a minute. The run kept writing, and the dataset status afterwards reports all 960 receipts with verified artifact hashes. All 20

`error` cells are the same capture-budget stop on the same arm and family. Twelve occurred before the incident, and the eight after it continue the same cell pattern, so the cause is the trace budget, not the disk.

- **What is committed.** The raw run tree (1.8 GB) stays in the session. Committed in `results/` are the public features, export manifest and generation log as `export-public.tar.xz`, the sha256 of every export file, the joined tables, the summaries, the audit and the dataset status. Restricted labels are not committed.

1.7. For the decisive run

1. Keep the exporter's field-size components, or define curve-level components. The pilot suggests curve identity drives `double-large-prime` outcomes, which grouped splits must respect.
2. Score `enumerate` and `root` separately or drop them from the pooled metric: they are deterministic at this scale.
3. Preregister the ecdlp endpoint beside completion.
4. Add the protocol's censored likelihood for work-to-event (time-to-event), not only the binary event at the cap.